List(s)

11 February 2025 20:11

A list in python is a mutable ordered collection of elements that can hold a mix of data types, including in intgers, strings, floats or even other lists. List are one of the most versatile data structures in Python.

Creating a List

A list is created using brackets [] And elements are separated by commas.

```
# empty list
my_list = []
# List of integers
num = [1,2,3,4,5,6,7,8,9]
# List of strings
names = ['Jiya', 'Aadya', 'Krita', 'Shrestha']
# Mixed list
mixed_list = [10, "Python", 3.14, True]
# nested list
nested_list = [[10, 15], [11, 12], [20,40]]
print(my_list)
print(num)
print(names)
print(mixed_list)
print(nested_list)
print(nested_list[0][0])
```

Output:

```
[]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
['Jiya', 'Aadya', 'Krita', 'Shrestha']
[10, 'Python', 3.14, True]
[[10, 15], [11, 12], [20, 40]]
10
```

Accessing Elements in a List

You can access elements using indexing (starting from 0) or negative indexing (starting from minus 1 for the last element).

```
 0        1        2        3
['Jiya', 'Aadya', 'Krita', 'Shrestha']
-4       -3       -2       -1
```

```
# empty list
my_list = []
# List of integers
num = [1,2,3,4,5,6,7,8,9]
# List of strings
names = ['Jiya', 'Aadya', 'Krita', 'Shrestha']
# Mixed list
mixed_list = [10, "Python", 3.14, True]
# nested list
nested_list = [[10, 15], [11, 12], [20,40]]
print(my_list)
print(num)
print(names)
```

```
print(mixed_list)
print(nested_list)
print(nested_list[0][0])
print("First name:",names[0])
print("Last name:",names[-1])
print("Names at index 1-2:", names[1:3])
print('Even:', num[1::2])
# print('Odd:', num[0::2])
print('Odd:', num[::2])
print('Reverse num[]:',num[-1::-1])
```

Output:

```
[]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
['Jiya', 'Aadya', 'Krita', 'Shrestha']
[10, 'Python', 3.14, True]
[[10, 15], [11, 12], [20, 40]]
10
First name: Jiya
Last name: Shrestha
Names at index 1-2: ['Aadya', 'Krita']
Even: [2, 4, 6, 8]
Odd: [1, 3, 5, 7, 9]
Reverse num[]: [9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Modifying a List

Since lists are mutable, we can change their elements.

```
print(fruits)
fruits[3] = 'Watermelon'
print(fruits)
```

Output:

```
['Apple', 'Mango', 'Orange', 'Banana', 'Grapes', 'Strawberry']
['Apple', 'Mango', 'Orange', 'Watermelon', 'Grapes', 'Strawberry']
```

Adding Elements to a List

1) **append() => Adds an element to the end of the list.**
2) **insert(index, element) => Inserts an element at a specific position.**
3) **extend(iterable) => Extends the list with multiple elements.**

Example:

```
>>> num = []
>>> num
[]
>>> num.append(10)
>>> num
[10]
>>> num.append(100)
>>> num
[10, 100]
>>> num.insert(1, 50)
>>> num
[10, 50, 100]
>>> num.insert(10, 50)
>>> num
```

```
[10, 50, 100, 50]
>>> num.insert(10, 50)
>>> num
[10, 50, 100, 50, 50]
>>> num.append([2, 4, 6])
>>> num
[10, 50, 100, 50, 50, [2, 4, 6]]
>>> num.extend([30, 40, 50, 60])
>>> num
[10, 50, 100, 50, 50, [2, 4, 6], 30, 40, 50, 60]
 0  1   2   3  4     5      6  7  8  9

>>>          num[5][0] => 2
```

Remove Elements from a List

1) remove(value) => Removes the first occurrence of the specified value.
2) pop(index) => It removes and returns the element at the given index (default is last.)
3) del list[index] => Deletes an element by index.
4) clear() => Removes all the elements from the list.

Examples:

```
>>> num
[10, 50, 100, 50, 50, [2, 4, 6], 30, 40, 50, 60]
>>> num
[10, 50, 100, 50, 50, [2, 4, 6], 30, 40, 50, 60]
>>> num.remove(50)
>>> num
[10, 100, 50, 50, [2, 4, 6], 30, 40, 50, 60]
>>> num.pop()
60
>>> num
[10, 100, 50, 50, [2, 4, 6], 30, 40, 50]
>>> num.pop(4)
[2, 4, 6]
>>> num
[10, 100, 50, 50, 30, 40, 50]
>>> del num[20]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list assignment index out of range
>>> del num[-1]
>>> num
[10, 100, 50, 50, 30, 40]
>>> num.pop(2)
50
>>> num
[10, 100, 50, 30, 40]
>>> num.clear()
>>> num
[]
>>>
```

Loop through a List

```
fruits = ['Apple', 'Mango', 'Orange', 'Banana', 'Grapes', 'Strawberry']
for fruit in fruits:
    print(fruit)
```

Output:

```
Apple
Mango
Orange
Banana
Grapes
Strawberry
```

```
matrix = [[10, 12, 14], [16, 18, 20], [22, 24, 26]]
```

	0	1	2
0	10	12	14
1	16	18	20
2	22	24	26

```
matrix = [[10, 12, 14], [16, 18, 20], [22, 24, 26]]
for row in matrix:
    print(row)
```

Output:

```
[10, 12, 14]
[16, 18, 20]
[22, 24, 26]
```

Accessing each element in the matrix separately

```
matrix = [[10, 12, 14], [16, 18, 20], [22, 24, 26]]
for row in matrix:
    for ele in row:
        print(f'\t{ele}',end = '')
    print() # to change the line
```

List Comprehension:
A compact way to create list:

```
squares = [x**2 for x in range(1,11)]
print(squares)
even_nums = [x for x in range(1, 21) if x % 2 == 0]
print(even_nums)
```

Output:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Sorting and reversing a List

1. **sort() => Sort the list in ascending order(Modifies the original list)**
2. **sorted() => Returns a new sorted list.**

method generates a list of random values,
range for random value.
Number of values.

```
import random as R
rand_num = R.sample(range(1, 100), 10)
print("Sorting in Ascending order:")
print("Before sorting: ",rand_num)
```

```
rand_num.sort()
print("After sorting:",rand_num)
# Soring in Descending order
rand_num = R.sample(range(1, 100), 10)
print("Sorting in Decending order:")
print("Before sorting: ",rand_num)
rand_num.sort(reverse=True)
print("After sorting:",rand_num)
```

Output:

```
Sorting in Ascending order:
Before sorting:  [52, 50, 35, 45, 95, 77, 63, 36, 67, 54]
After sorting: [35, 36, 45, 50, 52, 54, 63, 67, 77, 95]
Sorting in Decending order:
Before sorting:  [5, 33, 45, 35, 20, 42, 73, 52, 64, 83]
After sorting: [83, 73, 64, 52, 45, 42, 35, 33, 20, 5]
```

Another way of generating random list of 10 values

```
import random as R
rand_num = [R.randint(1,100) for _ in range(11)]
li = sorted(rand_num)
print(rand_num)
print(li)
```

list is not modified, sorted() method returns another sorted list.

Ascending and Descending order:

	0	1	2	3	4	5	6	7
rand_num =	91	2	61	32	45	28	9	16
dli =	2	9	16	28	32	45	61	91

```
import random as R
rand_num = [R.randint(1,100) for _ in range(11)]
li = sorted(rand_num)
print(rand_num)
print(li)
print()
dli = sorted(rand_num, reverse= True)
print(rand_num)
print(dli)
```

Output:

```
[84, 9, 57, 34, 55, 2, 20, 26, 88, 80, 33]
[2, 9, 20, 26, 33, 34, 55, 57, 80, 84, 88]

[84, 9, 57, 34, 55, 2, 20, 26, 88, 80, 33]
[88, 84, 80, 57, 55, 34, 33, 26, 20, 9, 2]
```

len(list)=> Returns the length of the list, meaning number of elements in the list.
count(value)=> Returns the count of occurrences of a value.
index(value)=> Returns the index of the first occurrence of a value.
copy()=> Returns a shallow copy of the list.

Example:

```
import random as R
rand_num = [R.randint(1,10) for _ in range(30)]
print(rand_num)
print(len(rand_num))
print(rand_num.count(2))
print(rand_num.index(3))
copy_rand_num = rand_num.copy()
print(copy_rand_num)
```

```
copy_rand_num[0] = 80
print(rand_num)
print(copy_rand_num)
```

Conclusion:
Python lists are powerful flexible and easy to use. They allow operations like adding, removing, sorting, reversing and filtering elements efficiently.

Programs:

1. WAP to input a String and display the frequency of vowels and consonant in each word. For example:

   Input: Enter any sentence: Mastering Information Technology
   Output:
   Mastering has 3 vowels and 6 consonants.
   Information has 5 vowels and 6 consonants.
   Technology has 3 vowels and 7 consonants.

2. **A non-palindrome word can be made a palindrome word just by adding reverse of the word with the original word. WAP to accept a non-palindrome word and display the new word after making it palindrome.**
   Input: CBSE
   OUTPUT: CBSEESBC

   INPUT: MADAM
   Output: Input is palindrome word.

3. **WAP to store 20 numbers in a list and calculate the sum of even numbers and odd numbers separately. And print the sums.**

Home work:

P6.1 Write a program that initializes a list with ten random integers and then prints four lines of output, containing

- Every element at an even index.
- Every even element.
- All elements in reverse order.
- Only the first and last element.

count = ~~0 1 2 3 4 5 6~~ 7 10

N = ~~10 20 25 30 15 20 18 25 30~~ 15, 35

li = [10, 20, 25, 30, 15, 18, 35, ...

P6.2 Write a program that reads numbers and adds them to a list if they aren't already contained in the list. When the list contains ten numbers, the program displays the contents and quits.

P6.3 Write a program that adds all numbers from 2 to 10,000 to a list. Then remove the multiples of 2 (but not 2), multiples of 3 (but not 3), and so on, up to the multiples of 100. Print the remaining values.

P6.4 Write list functions that carry out the following tasks for a list of integers. For each function, provide a test program.

a. Swap the first and last elements in the list.

b. Shift all elements by one to the right and move the last element into the first position. For example, 1 4 9 16 25 would be transformed into 25 1 4 9 16.

c. Replace all even elements with 0.

d. Replace each element except the first and last by the larger of its two neighbors.

e. Remove the middle element if the list length is odd, or the middle two elements if the length is even.

f. Move all even elements to the front, otherwise preserving the order of the elements.

g. Return the second-largest element in the list.

h. Return true if the list is currently sorted in increasing order.

i. Return true if the list contains two adjacent duplicate elements.

j. Return true if the list contains duplicate elements (which need not be adjacent).

■ **P6.5** Modify the `largest.py` program in Section 6.3 to mark both the smallest and the largest elements.

■■ **P6.6** Write a function `sumWithoutSmallest` that computes the sum of a list of values, except for the smallest one, in a single loop. In the loop, update the sum and the smallest value. After the loop, subtract the smallest value from the sum and return the difference.

■ **P6.7** Write a function `removeMin` that removes the minimum value from a list without using the `min` function or `remove` method.

■■ **P6.8** Compute the *alternating sum* of all elements in a list. For example, if your program reads the input

1 4 9 16 9 7 4 9 11

then it computes

$$1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2$$

■ **P6.9** Write a function that reverses the sequence of elements in a list. For example, if you call the function with the list

1 4 9 16 9 7 4 9 11

then the list is changed to

11 9 4 7 9 16 9 4 1

■ **P6.10** Write a function that implements the algorithm developed in Section 6.6.

■■ **P6.11** Write a function `def equals(a, b)` that checks whether two lists have the same elements in the same order.

■■ **P6.12** Write a function `def sameSet(a, b)` that checks whether two lists have the same elements in some order, ignoring duplicates. For example, the two lists

1 4 9 16 9 7 4 9 11

and

11 11 7 9 16 4 1

would be considered identical. You will probably need one or more helper functions.

■■■ **P6.13** Write a function `def sameElements(a, b)` that checks whether two lists have the same elements in some order, with the same multiplicities. For example,

1 4 9 16 9 7 4 9 11

and

11 1 4 9 16 9 7 4 9

would be considered identical, but

1 4 9 16 9 7 4 9 11

and

11 11 7 9 16 4 1 4 9

would not. You will probably need one or more helper functions.